A Strategy Language for Controlled Proof Search

Romain Sidhoum Simon Robillard David Delahaye

LIRMM, Univ. Montpellier, CNRS, Montpellier, France
Firstname.Lastname@lirmm.fr

This paper introduces the strategy language of Pgeon, a meta-prover with a clear separation between inference rules and proof search. We give the semantics of strategies as functions over proof states, and of the operators that are used to combine them, allowing for sequential composition, choice, repetition and interleaving of strategies. This language is designed to handle the challenge of fair proof search in semi-decidable logics, where simple depth-first exploration of the proof space is not guaranteed to achieve completeness. We showcase the expressiveness and effectiveness of the approach through case studies in first-order and modal logics.

1 Introduction

In automated reasoning, the completeness of a logical calculus does not guarantee that a proof will be found. In semi-decidable logics, some rules may generate infinitely many successors, and unfair exploration may indefinitely postpone relevant inferences, preventing the discovery of valid proofs. The completeness of the calculus is sometimes referred to as *static completeness*, while *dynamic completeness* describes the property of a prover, i.e., a proof search algorithm applied to a given calculus, to find a proof for any theorem [8].

These issues are particularly visible in systems that separate inference rules from proof-search strategies. While rules describe admissible transformations, strategies determine how they are applied during search, making the design of effective exploration policies non-trivial.

Pgeon [7] is a framework that separates logical rules from proof-search strategies and compiles them into executable provers. This separation highlights the impact of strategy design: the same set of rules may lead to different outcomes depending on how they are explored.

We introduce a formal strategy language for Pgeon, together with a compositional semantics that models proof search as the enumeration of proof states. Strategies are interpreted as functions mapping states to (possibly infinite) streams of successors, capturing both deterministic and non-deterministic behavior. The language provides combinators for sequential composition, choice, repetition, and fair interleaving of strategies.

We illustrate the approach through case studies in first-order and modal logics, showing how naive strategies may fail to terminate and how fair combinators enable systematic exploration of the search space. More generally, the framework provides a basis for reasoning about proof-search procedures at an abstract level, independently of implementation details.

Related Work Our strategy language relates to work on compositional stream processing, monadic nondeterminism, and strategy languages.

Monoidal stream semantics [3] models stream processors within a symmetric monoidal category, providing a principled account of compositionality where complex transformations are built from sim-

Submitted to:
LFMTP'26

pler ones. Similarly, functional reactive programming (FRP) [4] treats stream transformers as compositional entities combined via structured interfaces. However, these approaches are typically deterministic, whereas our setting is inherently nondeterministic and allows multiple (possibly infinite) results, requiring explicit control over exploration.

The treatment of nondeterministic computations over streams is closely related to the LogicT framework [6], which provides a monadic encoding of backtracking with both biased and fair search. Sequential composition in our language corresponds to Kleisli composition, while our choice operators capture biased and fair exploration. In contrast to LogicT, which provides an operational abstraction, we reify computations as lazy streams and introduce an explicit algebra of strategy combinators.

Our work is also related to tactic languages such as $\mathscr{L}_{tac}$ [2], used in the prover Rocq, where tactics act as transformations over proof states with failure and backtracking. While the analogy is structural, tactic languages typically provide an operational notion of search. In our approach, the search space is represented explicitly as a stream, and exploration strategies are expressed compositionally.

Rewriting-based systems such as Maude [1] similarly model computation as nondeterministic transitions and provide strategy languages for controlling rule application. However, rewriting logic is fundamentally relational, whereas our approach is functional and stream-based, reifying the search space as a first-class object and enabling compositional reasoning about fairness and enumeration.

2 Pgeon

Pgeon [7] generates automated theorem provers from declarative specifications of logical calculi. It is intended to generate provers based on the tableaux method. The specification of a prover defines the syntax, inference rules, and a strategy that describes how rules are applied. From this specification, an executable prover is produced.

This separation allows different proof-search behaviors to be defined over the same calculus, as the rules specify admissible steps while strategies control their exploration. In particular, rule applications are inherently non-deterministic, as they may match multiple formulas, branches, or terms, potentially generating infinitely many successors.

In Pgeon, this search policy is expressed by a strategy. Strategies control the order of rule applications, the exploration of alternatives, and the use of backtracking, making them a central component of the generated prover. The strategy language studied in Section 3 abstracts away from the concrete implementation of this mechanism. We treat a rule application as a function from proof states to streams of successor states. The stream represents the possible outcomes of applying the rule, ordered according to the intended exploration policy. This view makes non-determinism explicit since all possible outcomes are enumerated rather than implicitly explored. A deterministic rule returns a singleton stream, a failing rule returns the empty stream, and a non-deterministic rule returns a stream with several successors. A rule may yield an infinite stream of successors.

This abstraction is useful because it allows strategy operators to be defined compositionally. Sequential composition, choice, repetition, and fair interleaving can all be described as operations on streams of proof states. The resulting semantics does not depend on the stack discipline or continuation representation used by particular implementations.

The need for such a semantics it becomes clear when considering fairness. Suppose a strategy repeatedly applies a universally quantified rule before considering other instances. Even if a contradiction can be found after a finite number of different instantiations, a depth-first strategy may keep generating redundant or irrelevant instances forever. The problem is not that the calculus is incomplete, but that the

strategy unfairly gives unbounded priority to one part of the search space.

For example, consider the clause set:

$$\Gamma = \{\ \forall x.\neg P(x) \vee \neg Q(x),\ P(a),\ P(b),\ Q(b)\ \}$$

A closing derivation requires selecting useful instances of the universal formula and combining them with the atomic facts already present in the branch. However, repeated instantiation with $x \mapsto a$ produces an infinite expansion without progress. The search then follows an infinite path although the contradiction is reachable in the proof space. This phenomenon can be sketched as follows:

$$\dfrac{\dfrac{\Gamma}{\Gamma, \neg P(a)} \quad \dfrac{\dfrac{\Gamma}{\Gamma, \neg Q(a)}\ \forall, \vee, x \mapsto a}{\dfrac{\Gamma, \neg Q(a), \neg P(a)}{\vdots} \quad \dfrac{\Gamma, \neg Q(a), \neg Q(a)}{\vdots}}\ \forall, \vee, x \mapsto a}$$

This motivates the fair strategy combinators introduced in the rest of the paper. Rather than relying on depth-first exploration alone, the language provides operators that interleave the results of several strategies. Interleaving ensures that if a successor state occurs at some finite position in one of the component searches, then it also occurs at some finite position in the combined search, so no branch that can produce a result is postponed indefinitely. In this way, fair composition prevents one infinite branch from starving the exploration of another.

3 Strategy Language

We formalize proof search as the transformation of proof states. A proof state represents a partially constructed proof tree, whose open branches correspond to sets of formulas to be closed. Applying an inference rule expands the proof tree, producing new proof states.

Let Σ be the set of proof states. In general, a proof state can be seen as a tree of formulas, whose branches are either open or closed. The precise structure of Σ is left abstract, as our framework is parametric in the underlying proof system.

Rather than modeling proof search as a relation on states, we adopt a view where a strategy enumerates all possible outcomes of its application. Formally, we write $\mathrm{Str}(\Sigma)$ for the set of (finite or infinite) streams over Σ, and define a strategy as a function:

$$s : \Sigma \to \mathrm{Str}(\Sigma)$$

Given a state $\sigma \in \Sigma$, the stream $s(\sigma)$ represents all possible successor states obtained by applying s to σ. The empty stream denotes failure, a singleton stream denotes deterministic success, and longer streams represent multiple possible successor states (non-deterministic branching).

Each inference rule r induces a primitive strategy:

$$[\![r]\!] : \Sigma \to \mathrm{Str}(\Sigma)$$

which returns all proof states obtained by applying r for each admissible match of premises. Depending on the rule, the matched premises may either be removed from the state, or preserved. In the latter case, it is often useful to apply the rule exhaustively to all applicable matches. To this end, we introduce a derived operator $r!$, which applies r to all admissible matches in a given state. Its semantics is given by:

$$[\![r!]\!] : \Sigma \to \mathrm{Str}(\Sigma)$$

where $[\![r!]\!](t)$ returns the unique state obtained by exhaustively applying r to all applicable premises in t, if at least one application is possible, and the empty stream otherwise.

Stream operators We assume the following operators on streams:

- The stream operator id such that $\mathrm{id}(t) = \langle t \rangle$
- For a function $f : \Sigma \to \mathrm{Str}(\Sigma)$ and a stream X, $X \gg\!= f$ is the stream obtained by applying f to each element of X and concatenating the results.
- Given a stream of streams $S = \langle S_0, S_1, S_2, \ldots \rangle$ with $S_i = \langle \sigma_{i,0}, \sigma_{i,1}, \ldots \rangle$, we define $\triangle(S)$ the stream that contains every $\sigma_{i,j}$, ordered such that $\sigma_{i,j}$ appears before $\sigma_{i',j'}$ if
  (a) $i + j < i' + j'$, or
  (b) $i + j = i' + j'$ and $i < i'$.

  This construction ensures that every element $\sigma_{i,j}$ appears after finitely many elements in $\triangle(S)$. Thus, if a result appears at a finite depth in any component stream, it appears at a finite depth in the interleaved stream.

Strategy combinators Strategies are built from primitive rules (r) using the following grammar:

$$s \quad ::= \quad r \mid r! \mid s\,; s \mid s \,\|\, s \mid s \,\&;\, s \mid s \,\&|\, s \mid s^*$$

Let $s, s_1, s_2 : \Sigma \to \mathrm{Str}(\Sigma)$. The semantics of the combinators is defined as follows.

- Sequential composition:
$$[\![s_1\,; s_2]\!](t) = [\![s_2]\!] \gg\!= [\![s_1]\!](t)$$
This corresponds to applying s_1 to t, and then applying s_2 to each resulting state.
- Left-biased choice:
$$[\![s_1 \,\|\, s_2]\!](t) = \begin{cases} [\![s_1]\!](t) & \text{if } [\![s_1]\!](t) \neq \emptyset, \\ [\![s_2]\!](t) & \text{otherwise} \end{cases}$$
This operator models deterministic choice with fallback.
- Fair sequential composition:
$$[\![s_1 \,\&;\, s_2]\!](t) = \triangle([\![s_1]\!](t) \gg\!= [\![s_2]\!])$$
Unlike $\|$, this operator interleaves the exploration of all intermediate results, ensuring that no branch is indefinitely delayed. If $s_2(s_1(t))$ can produce a result in finite steps, then $s_1 \,\&;\, s_2$ will eventually produce it.
- Fair choice:
$$[\![s_1 \,\&|\, s_2]\!](t) = \triangle(\langle [\![s_1]\!](t), [\![s_2]\!](t) \rangle)$$
This operator fairly interleaves the results of both strategies.
- Repetition:
$$s^* = \mu X.\, \mathrm{id} \,\&;\, (s \,\&;\, X)$$
This defines the closure under repeated application of s, with fairness ensuring that all finite iteration depths are explored. Unlike the usual Kleene star based on sequential composition, this definition uses fair sequential composition. As a result, the exploration of derivations is interleaved across all iteration depths, rather than proceeding depth-first. The fixpoint is understood as the least fixpoint on strategies ordered by prefix inclusion of their output streams.

The operators ; and $\|$ induce a depth-first exploration of the search space, as they process results sequentially and may indefinitely delay alternative branches. The operators &; and &| use diagonalization to ensure a fair exploration, guaranteeing that every branch that produces results is eventually explored.

4 Case Studies

4.1 First-Order Logic (LK)

We illustrate the expressiveness of the strategy language on a tableau calculus for classical first-order logic (LK). Inferences are classified into four families: non branching rules α, branching rules β, universal rules γ, and existential rules δ. The rules are given in Figure 1. This calculus uses a destructive closure rule $\odot$: the closing substitution is applied to all branches, potentially forbidding inferences that would have previously been possible. The closing rule is the only source of instantiation, i.e., there is no γ rule that can provide an instance $P(t)$ of some universally quantified formula.

$$\frac{P \qquad \neg P}{\odot}\,\odot \qquad \frac{\neg\neg P}{P}\,\alpha\neg\neg$$

$$\frac{P \wedge Q}{P, Q}\,\alpha\wedge \qquad \frac{\neg(P \vee Q)}{\neg P, \neg Q}\,\alpha\neg\vee$$

$$\frac{\neg(P \Rightarrow Q)}{P, \neg Q}\,\alpha\neg\Rightarrow \qquad \frac{P \vee Q}{P \mid Q}\,\beta\vee$$

$$\frac{\neg(P \wedge Q)}{\neg P \mid \neg Q}\,\beta\neg\wedge \qquad \frac{P \Rightarrow Q}{\neg P \mid Q}\,\beta\Rightarrow$$

$$\frac{\exists x.P(x)}{P(f(X_1,\ldots,X_n))}\,\delta\exists \qquad \frac{\neg\forall x.P(x)}{\neg P(f(X_1,\ldots,X_n))}\,\delta\neg\forall$$

$$\frac{\forall x.P(x)}{P(X)}\,\gamma\forall \qquad \frac{\neg\exists x.P(x)}{P(X)}\,\gamma\neg\exists$$

$$\frac{P \qquad \neg Q}{\odot, \text{ apply } \sigma \text{ to the tree if } P \text{ and } Q \text{ are unifiable and } \sigma = \mathrm{mgu}(P,Q)}\,\odot$$

Figure 1: LK Rules for First-Order Classical Logic

A common heuristic is to separate rule applications into phases: First, perform all applications of α and δ rules, then applications of β rules. Interleave this process with instantiations of universal formulas. A naive strategy would consist in saturating the branch and repeatedly applying γ: $(\alpha \,\|\, \delta \,\|\, \beta \,\|\, \gamma)^*$.

The main difficulty lies in the interaction between closure and the rest of the proof search. Failing to consider certain closure choices may delay the discovery of useful instantiations. An effective strategy must therefore ensure that all relevant closure-induced substitutions are eventually explored.

To address this issue, we separate closure from the rest of the search and combine them using fair composition:

$$(\odot^* \,\&;\, (\alpha \,\|\, \delta \,\|\, \beta \,\|\, \gamma!)^*)^*$$

The sub-strategy $\odot^*$ enumerates all finite streams of closure applications, thereby exploring different possible instantiations induced by unification. The sub-strategy $(\alpha \,\|\, \delta \,\|\, \beta \,\|\, \gamma!)^*$ performs saturation using the remaining rules. Because $\|$ is left-biased, it can be used to prioritize certain rules. The fair composition operator &; interleaves these two processes, ensuring that closure attempts are distributed

across all depths of the search. As a result, the strategy avoids committing prematurely to a particular instantiation: if a closing derivation exists, it will eventually be explored.

4.2 Modal Logic (S4)

We now consider a second case study based on propositional modal logic S4 [5]. We consider a standard classification of tableau rules for propositional S4. In addition to the usual proposition rules (α and β), we use the three modal rules described in Figure 2.

$$\frac{P}{}\ \theta^{a} \qquad \frac{\Box P}{\Box P; P}\ T \qquad \frac{\Box X; \neg\Box P}{\Box X; \neg P}\ S4^{b}$$

[a] This rule operates on a single branch. In pgeon, the tableau is represented as a forest of branches, enabling local transformation.

[b] This rule is non-local, as it requires access to the entire branch, unlike the other rules which are applied to individual formulas.

Figure 2: S4 Rules for Propositional Modal Logic

The main source of non-determinism is the rule θ, that allows arbitrary removal of formulas from the branch. θ is necessary to expose formulas to the modal rule $S4$ by removing blocking formulas so that $S4$ becomes applicable, but it may also discard formulas that are needed later to close the branch. An uncontrolled use of θ may therefore lead to dead ends in the search space.

The key observation is that θ should not be used arbitrarily, but only in order to enable applications of $S4$. We capture this idea by grouping θ and $S4$ into a combined strategy: $\theta^* \,\&;S4$. We combine this construction with the other rules to obtain the following strategy:

$$((\alpha \,\|\, \beta \,\|\, T)^*;(\theta^* \,\&;S4))^*$$

At each iteration, the strategy first saturates using α, β, and T, which are terminating using loop checking. If no such rule is applicable, it applies the combined strategy which explores all ways of applying $S4$ with the removal of some formulas. The use of fair composition ensures that different choices of formulas to discard are explored without starvation. If a successful application of $S4$ requires removing a specific subset of formulas, this configuration will eventually be reached. If closing the tableau additionally requires not discarding a specific subset of formulas, this configuration will also be reached.

5 Conclusion

This work introduced a formal strategy language for controlling proof search in tableau-based automated theorem proving, as implemented in Pgeon. By modeling strategies as functions from proof states to streams of successor states, the framework makes non-determinism explicit and enables a compositional semantics for combining search behaviors. The language provides both biased and fair combinators, with fairness achieved through interleaving constructions that prevent the starvation of branches in potentially infinite search spaces. Through case studies in first-order and modal logics, we demonstrated

how naive depth-first strategies may fail to find proofs despite the completeness of the underlying calculus, and support dynamic completeness by enabling systematic exploration of the search space. Overall, this approach offers a principled and modular foundation for specifying, analyzing, and reasoning about proof-search procedures independently of implementation details, while addressing fundamental challenges inherent to semi-decidable logics.

References

[1] Clavel, M., Durán, F., Eker, S., Lincoln, P., Martí-Oliet, N., Meseguer, J., Talcott, C.: All about maude-a high-performance logical framework: how to specify, program, and verify systems in rewriting logic, vol. 4350. Springer (2007)

[2] Delahaye, D.: A tactic language for the system coq. In: International Conference on Logic for Programming Artificial Intelligence and Reasoning. pp. 85–95. Springer (2000)

[3] Di Lavore, E., de Felice, G., Román, M.: Monoidal streams for dataflow programming. In: Proceedings of the 37th Annual ACM/IEEE Symposium on Logic in Computer Science. pp. 1–14 (2022)

[4] Elliott, C., Hudak, P.: Functional reactive animation. In: Proceedings of the second ACM SIGPLAN international conference on Functional programming. pp. 263–273 (1997)

[5] Goré, R.: Tableau methods for modal and temporal logics. In: Handbook of tableau methods, pp. 297–396. Springer (1999)

[6] Kiselyov, O., Shan, C.c., Friedman, D.P., Sabry, A.: Backtracking, interleaving, and terminating monad transformers: (functional pearl). ACM SIGPLAN Notices **40**(9), 192–203 (2005)

[7] Sidhoum, R., Robillard, S., Delahaye, D.: Pgeon: Generating tableau-based provers from declarative specifications of logical calculi. In: International Joint Conference on Automated Reasoning (IJCAR) (2026), to appear

[8] Waldmann, U., Tourret, S., Robillard, S., Blanchette, J.: A comprehensive framework for saturation theorem proving. In: International Joint Conference on Automated Reasoning. pp. 316–334. Springer (2020)